

Pose, Velocity and the Differential Drive

How Two Wheels Become a Trajectory

Prof. Anis Koubaa Dr. Asem Ibrahim Alalwan

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 4, Lecture A



Lecture Overview

Roadmap

Last week `/cmd_vel` was a *topic* — a name, a type, and a contract. Today it is *two numbers with units*. We define **pose** and **velocity** in the plane, derive the **differential-drive model** that turns two wheel speeds into those two numbers and back, integrate it to get **odometry**, and then account for every reason the answer is wrong.

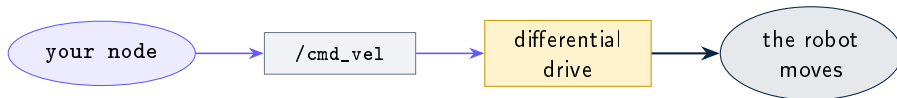
- 1 Pose in the Plane
- 2 Velocity
- 3 The Differential-Drive Model
- 4 From Velocity to Position
- 5 Wheel Odometry and Its Errors
- 6 Open Loop, and What Comes Next

Part 1

Pose in the Plane

- 1 Pose in the Plane
- 2 Velocity
- 3 The Differential-Drive Model
- 4 From Velocity to Position
- 5 Wheel Odometry and Its Errors
- 6 Open Loop, and What Comes Next

From the Graph to the Geometry



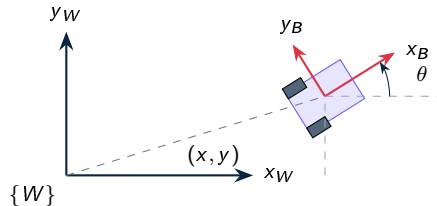
Week 3 stopped here:
a type and a contract

Week 4 starts here:
what the numbers mean

A student who has finished Week 3 can publish a Twist and make a turtle move. They cannot yet answer: *how far did it go? Which way is it facing now? What should I send to travel exactly one metre?* Those three questions are this week.

The message was the interface. The model is what makes the interface predictable.

Two Frames, and Why Both Are Needed



- $\{W\}$ — the **world frame**. Fixed. Where the robot *is*.
- $\{B\}$ — the **body frame**. Rides on the robot, x_B out of its nose. What the robot can *do*.

Why not one frame

A robot cannot be commanded in $\{W\}$: “move north” means nothing to a motor. It can only be commanded in $\{B\}$: “drive forward, turn left.” Every position it reports is in $\{W\}$; every command it accepts is in $\{B\}$. The model in this lecture is the translation between the two.

Definition: Pose

Pose

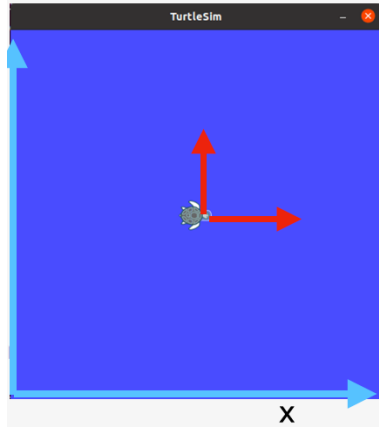
The **pose** of a planar mobile robot is the triple (x, y, θ) : the position of the body origin expressed in the world frame, and the angle from x_W to x_B , positive counter-clockwise. Three numbers, three degrees of freedom — and no more, because the robot cannot leave the floor.

Symbol	Unit	Notes
x, y	metres	In turtlesim, a made-up unit on an 11×11 canvas.
θ	radians	Never degrees, anywhere in ROS. Wrapped to $(-\pi, \pi]$.

In the field

Degrees appear in exactly one place in this course: on a slide, for a human. The moment a number enters a message it is radians. Mixing the two is the single most common bug in a first motion node, and it fails quietly — the robot turns, just not by the amount you asked for.

The Pose Is a Frame Painted on the Robot



The turtle carries its own axes: red is x_B (forward) and y_B (left); blue is the world frame of the canvas.

Every command is written in red; every position is reported in blue.

Capture from A. Koubaa, *ROS for Beginners: Basics, Motion and OpenCV* (2020). The turtlesim window is unchanged under ROS 2 Jazzy.

The Pose, as It Arrives

```
$ ros2 topic echo /turtle1/pose --once  
x: 6.175500869750977  
y: 7.55879020690918  
theta: 2.5055999755859375  
linear_velocity: 0.0  
angular_velocity: 0.0  
---
```

/turtle1/pose at about 60 Hz. Five floats: three are the pose, two are the velocity the simulator is currently applying. **theta is in radians** — 1.57 is a quarter turn, and it is the number your code must reason about.

Two Ways ROS 2 Hands You a Pose

`turtlesim/msg/Pose`

```
float32 x
float32 y
float32 theta
float32 linear_velocity
float32 angular_velocity
```

The angle is *given* to you.

`nav_msgs/msg/Odometry` (abridged)

```
string    child_frame_id
Point     position      x y z
Quaternion orientation   x y z w
Twist     twist
```

The angle is *encoded*, in four numbers.

Same quantity, two representations. Turtlesim publishes θ because it is a 2-D toy; every real robot publishes a **quaternion**, because one message type must also serve a drone, an arm and a legged robot.

You will write the conversion once, in session B, and use it every week until December.

Why Four Numbers? The Case for the Quaternion

On a floor, one angle is enough. Off the floor — a drone, a pan-tilt camera, an arm — an orientation is **an axis to spin about, and how far**: three numbers along the axis, one for the amount.

$(0, 0, 0, 1)$ is “no rotation”; a flat robot at heading θ is $(0, 0, \sin \frac{\theta}{2}, \cos \frac{\theta}{2})$.

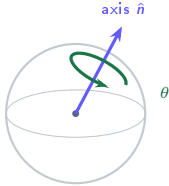
Representation	Numbers	What goes wrong
One angle θ	1	Nothing — until the robot leaves the plane.
Roll, pitch, yaw	3	Gimbal lock : nose straight up, and roll and yaw become the same knob.
Quaternion	4	Nothing. Four numbers, one constraint: $x^2 + y^2 + z^2 + w^2 = 1$.

Why it is also the numerically stable choice

Composing quaternions is multiply-and-add: no $\sin$, no $\cos$, no division by $\cos(\text{pitch})$ — the term that goes to zero, and takes the answer to infinity, exactly at gimbal lock. Accumulated drift is repaired in one line, $q \leftarrow q / \|q\|$, where a drifted 3×3 matrix must be re-orthonormalised. Hence tf2 stores quaternions and converts only at the edges.

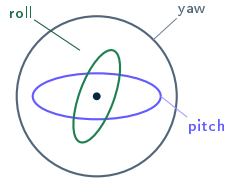
The Same Idea, Drawn: Axis, Angle, and Lock

What a quaternion stores



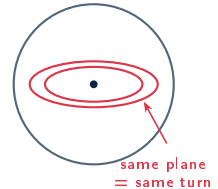
Which way to spin, and how far.

Three angles: free



Three rings in three planes: three separate knobs.

Three angles: locked



Pitch at 90° : two rings line up. Three knobs, two effects.

A quaternion has no rings to line up — which is why tf2 stores one, and why the conversion on the next slide is the only place you meet an angle.

Quaternion to Yaw — the One Formula You Need

For a robot flat on the floor, roll and pitch are zero and the quaternion (q_x, q_y, q_z, q_w) collapses to a single rotation about z:

$$\theta = \text{atan2}(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2))$$

```
from math import atan2
q = msg.pose.pose.orientation
yaw = atan2(2.0 * (q.w * q.z + q.x * q.y),
            1.0 - 2.0 * (q.y * q.y + q.z * q.z))
```

Why atan2 and not atan

`atan` returns a value in $(-\pi/2, \pi/2)$ and cannot tell forward from backward. `atan2` takes the two arguments separately, keeps their signs, and returns the full $(-\pi, \pi]$. A robot that drives correctly in two quadrants and backwards in the other two has this bug.

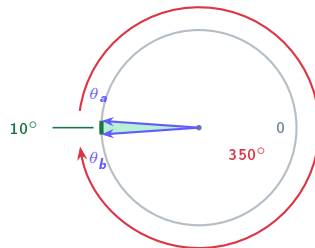
Angles Wrap, and That Is Where the Bugs Live

Two headings, $\theta_a = 175^\circ$ and $\theta_b = -175^\circ$. Subtract them and you get 350° . The robot needs to turn 10° .

$$\Delta\theta = \text{atan2}(\sin(\theta_b - \theta_a), \cos(\theta_b - \theta_a))$$

the **shortest** angular distance, always in $(-\pi, \pi]$, sign included.

Every angle in ROS lives on a circle, not a line. Two numbers that differ by 2π are the same heading, and no message ever tells you which representative you were handed.



red: 350° — what subtraction gives

green: 10° — what the robot must turn

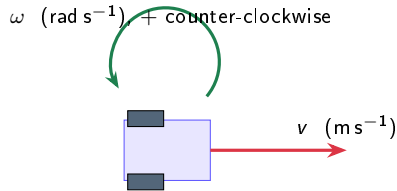
Where you will meet it, this week

The open-loop square in session B crosses $\pm\pi$ at its fourth corner. A robot that spins almost all the way round instead of turning the last quarter has this bug and nothing else.

Part 2 Velocity

- 1 Pose in the Plane
- 2 **Velocity**
- 3 The Differential-Drive Model
- 4 From Velocity to Position
- 5 Wheel Odometry and Its Errors
- 6 Open Loop, and What Comes Next

Linear and Angular Velocity, in the Body Frame



A planar robot has **three** degrees of freedom and a differential drive can command only **two** of them:

- v — how fast the body origin moves *along* x_B . Forward is positive.
- ω — how fast the body rotates about z_B . Left is positive.

v and ω are the entire vocabulary. Every motion in this course — every square, spiral, wall-follow and navigation goal — is a schedule of these two numbers.

Twist: Six Numbers, Two That Matter

```
Vector3 linear
  float64 x
  float64 y
  float64 z
Vector3 angular
  float64 x
  float64 y
  float64 z
```

Field	Differential drive
<code>linear.x</code>	v — used
<code>linear.y</code>	ignored — see below
<code>linear.z</code>	ignored (not a drone)
<code>angular.x</code>	ignored (roll)
<code>angular.y</code>	ignored (pitch)
<code>angular.z</code>	ω — used

Setting `linear.y` produces no error, no warning, and no sideways motion — the driver reads two fields and discards four. The type is over-general on purpose: it also serves an omnidirectional base, which uses `linear.y`.

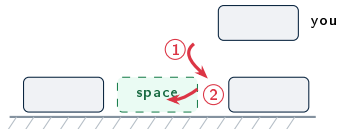
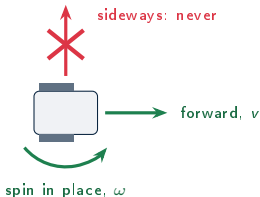
The message cannot tell you which fields your robot honours. The robot's documentation can.

Why the Missing Field Is a Law, Not an Oversight

The wheels roll and do not slide. That single physical fact is written:

$$\dot{x} \sin \theta - \dot{y} \cos \theta = 0 \quad \text{i.e. no component along } y_B, \text{ ever.}$$

A constraint on *velocities* that cannot be integrated into a constraint on *positions* is called **nonholonomic**. Its consequence is the one every driver knows:



The space is wide enough and one metre away. It still takes two moves.

Two knobs — and the third direction is not one of them.

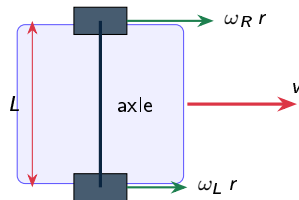
Parallel parking is not a hard problem because cars are badly designed. It is hard because $\dot{y}_B = 0$.

Part 3

The Differential-Drive Model

- 1 Pose in the Plane
- 2 Velocity
- 3 The Differential-Drive Model
- 4 From Velocity to Position
- 5 Wheel Odometry and Its Errors
- 6 Open Loop, and What Comes Next

The Mechanism



wheel radius r , wheel separation L

Two independently driven wheels on a common axle, plus a caster that carries weight and steers nothing. There is no steering mechanism at all: **the robot turns because the two wheels run at different speeds.**

In the field

TurtleBot 3 Burger, the platform for the rest of this course: $r = 0.033 \text{ m}$, $L = 0.160 \text{ m}$, $v_{\max} = 0.22 \text{ m s}^{-1}$, $\omega_{\max} = 2.84 \text{ rad s}^{-1}$. Four numbers that turn the next slides into arithmetic you can check.

Forward Kinematics: Wheels to Body

Each wheel's contact point moves at $r\omega_R$ and $r\omega_L$. The body origin sits halfway between them, so it moves at their average; the body rotates at their difference divided by the separation:

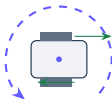
$$v = \frac{r(\omega_R + \omega_L)}{2}$$

$$\omega = \frac{r(\omega_R - \omega_L)}{L}$$



$$\omega_R = \omega_L$$

$v = 0$: a straight line



$$\omega_R = -\omega_L$$

$v = 0$: spin about the axle centre



$$\omega_L = 0$$

turn about the stopped wheel

Where this equation actually runs

Inside the robot's driver, once per control cycle, in the direction shown here — encoders to (v, ω) — to publish `/odom`. It runs in the *other* direction to turn your `/cmd_vel` into motor set-points. Both directions, every cycle, forever.

Inverse Kinematics: Body to Wheels

Solve the same pair for the wheel speeds — this is the direction your `Twist` takes:

$$\omega_R = \frac{2v + \omega L}{2r}$$

$$\omega_L = \frac{2v - \omega L}{2r}$$

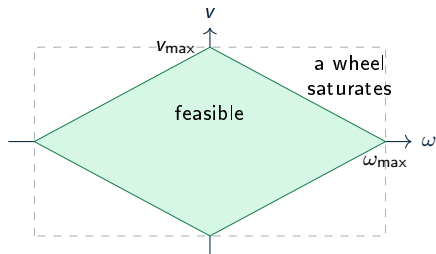
Worked example (Burger). You publish $v = 0.20$, $\omega = 1.0$:

$$\omega_R = \frac{0.40 + 0.16}{0.066} = 8.48 \text{ rad s}^{-1}, \quad \omega_L = \frac{0.40 - 0.16}{0.066} = 3.64 \text{ rad s}^{-1}$$

The Burger's motors top out near 6.4 rad s^{-1} . **The right wheel cannot do it.** The driver clamps it — and now v and ω are both wrong, silently, and the robot curves the wrong way.

A `Twist` within the published $v_{\max}$ and $\omega_{\max}$ can still be impossible. The limits are per wheel, not per field.

The Feasible Region Is a Diamond, Not a Box

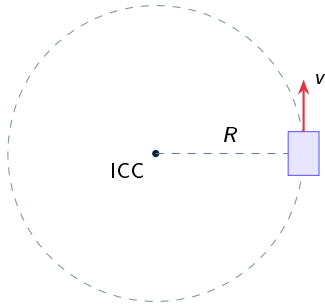


The dashed box is what the datasheet suggests. The shaded diamond is what the wheels can deliver, because $|2v \pm \omega L| \leq 2r\omega_{\text{wheel},\max}$ must hold for *both* wheels.

In the field

At $v = v_{\max}$ there is no headroom left for ω — which is why a robot that holds a line perfectly at full speed leaves it the moment a controller adds a correction. Week 5's controller has to give up speed to keep its turn.

The Instantaneous Centre and the Turning Radius



With v and ω both held constant the robot traces a circle. Its centre is the **instantaneous centre of curvature**, on the axle line, at

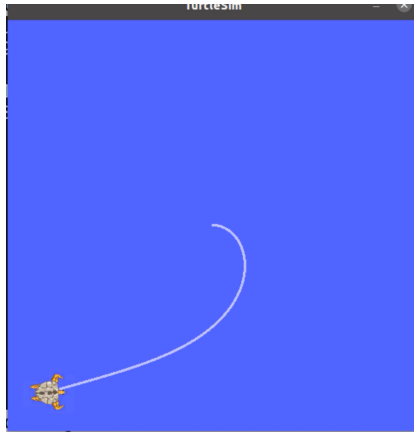
$$R = \frac{v}{\omega}$$

- $\omega \rightarrow 0 \Rightarrow R \rightarrow \infty$: a straight line.
- $v = 0 \Rightarrow R = 0$: spin in place.
- R inside the wheelbase: one wheel runs backwards.

A differential drive has no path primitive other than the arc. A straight line is an arc of infinite radius; a spin is an arc of zero radius.

Everything a two-wheeled robot can do is a sequence of arcs.

One Constant Twist, Held — and the Arc Appears



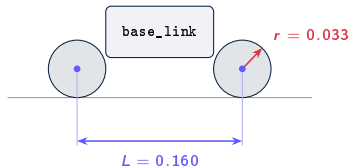
`linear.x` and `angular.z` both non-zero and both *unchanging*: the trail is a circular arc of radius $R = v/\omega$, exactly as the model predicts. **Nothing in the program curves — the geometry does.**

Capture from A. Koubaa, *ROS for Beginners: Basics, Motion and OpenCV* (2020).

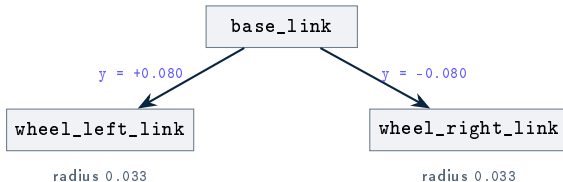
The Model Needs Two Numbers. Where Do They Live?

Every equation in this section has r and L in it. Neither is in your node: they are properties of the *robot*, and ROS 2 keeps them in one file — the **robot description**.

The robot, from behind



The same robot, as a tree of links and joints



L is not a number in the URDF. It is the *distance between two joint origins* — and r is the radius of the wheel geometry. Write the shape, and the model's constants fall out of it.

A Differential Drive, in Twelve Lines of URDF

The Burger's left half. The right half is the same file with one sign changed.

```
<robot name="burger">
  <link name="base_link"/>
  <link name="wheel_left_link">
    <visual><geometry>
      <cylinder radius="0.033" length="0.018"/>
    </geometry></visual>
  </link>
  <joint name="wheel_left_joint" type="continuous">
    <parent link="base_link"/> <child link="wheel_left_link"/>
    <origin xyz="0 0.080 0.023"/> <axis xyz="0 1 0"/>
  </joint>
</robot>
```

radius="0.033"

xyz="0 0.080 0"

type="continuous"

this is the r in $v = r(\omega_R + \omega_L)/2$

half of L . The right wheel sits at -0.080 , so $L = 0.160$

a joint that turns forever — a wheel, not a hinge

...and the World It Is Dropped Into

The URDF says what the robot *is*; the world says where it *starts* — the origin `/odom` counts from. One plugin reads r and L at run time:

```
<gazebo>
  <plugin filename="gz-sim-diff-drive-system"
    name="gz::sim::systems::DiffDrive">
    <left_joint>wheel_left_joint</left_joint>
    <right_joint>wheel_right_joint</right_joint>
    <wheel_separation>0.160</wheel_separation>
    <wheel_radius>0.033</wheel_radius>
  </plugin>
</gazebo>
```

What happens when the file and the robot disagree

The plugin is the inverse-kinematics box from three slides ago, once per cycle. Tell it $L = 0.160$ when the axle measures 0.155: the robot drives perfectly, and every turn it reports is scaled, forever. **Week 8 authors this file;** today it is only where r and L live.

Part 4

From Velocity to Position

- 1 Pose in the Plane
- 2 Velocity
- 3 The Differential-Drive Model
- 4 From Velocity to Position**
- 5 Wheel Odometry and Its Errors
- 6 Open Loop, and What Comes Next

Integrating the Model

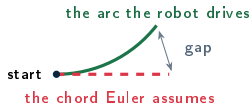
The model gives $\dot{x} = v \cos \theta$, $\dot{y} = v \sin \theta$, $\dot{\theta} = \omega$. Over one control step Δt with v and ω held constant:

Exact (arc), for $\omega \neq 0$

$$\theta' = \theta + \omega \Delta t$$

$$x' = x + \frac{v}{\omega} (\sin \theta' - \sin \theta)$$

$$y' = y - \frac{v}{\omega} (\cos \theta' - \cos \theta)$$



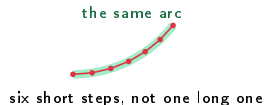
One step, exaggerated.

Euler (straight-line approximation)

$$x' = x + v \cos \theta \Delta t$$

$$y' = y + v \sin \theta \Delta t$$

$$\theta' = \theta + \omega \Delta t$$



The chords now ride inside the curve.

Euler drives the *chord*; the robot drives the *arc*. At 50 Hz and $\omega = 1 \text{ rad s}^{-1}$ the heading moves 1.1° a step, and the two are the same line.

The approximation is not the reason odometry drifts. The next section is.

The Update, as It Is Actually Written

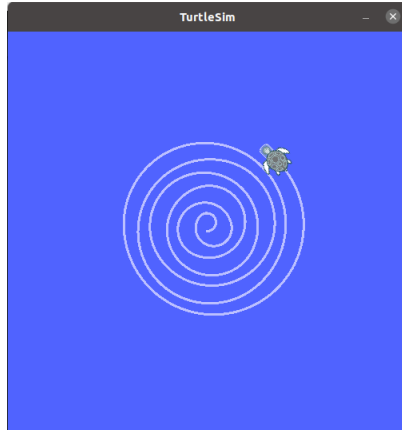
```
def integrate(x, y, th, v, w, dt):  
    x += v * cos(th) * dt  
    y += v * sin(th) * dt  
    th += w * dt  
    th = atan2(sin(th), cos(th))    # wrap to (-pi, pi]  
    return x, y, th
```

Four lines. Every mobile robot you will ever meet runs some version of them, tens of times a second, from the moment it powers on.

The wrap line is not cosmetic

Without it θ grows without bound and loses precision in its low bits. `atan2(sin, cos)` is the idiomatic wrap — two trig calls that remove a bug which only shows up in long runs, which is to say in the demo and not in the lab.

The Trail Is the Integral



A spiral is not a new primitive. It is ω held constant while v rises linearly — $R = v/\omega$ grows with every turn. **The picture is what those four lines of integration produce when you plot every step.**

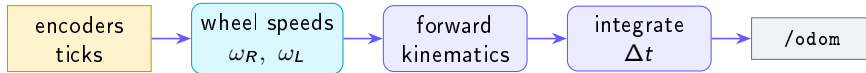
Capture from A. Koubaa, *ROS for Beginners: Basics, Motion and OpenCV* (2020).

Part 5

Wheel Odometry and Its Errors

- 1 Pose in the Plane
- 2 Velocity
- 3 The Differential-Drive Model
- 4 From Velocity to Position
- 5 Wheel Odometry and Its Errors**
- 6 Open Loop, and What Comes Next

What Produces /odom

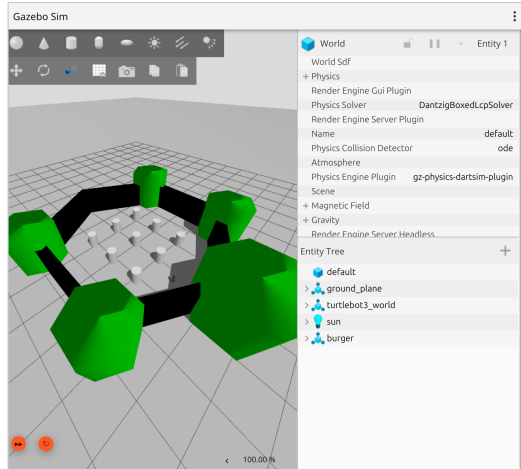


Odometry is the estimate of pose obtained by integrating measured wheel motion. It is not a measurement of position. Nothing in that chain ever observes where the robot is — it observes how much the *wheels* turned and assumes the rest.

In the field

It is nonetheless the most useful signal on the robot: available at 30–50 Hz, never missing, never occluded, and locally very accurate. Everything from Week 5's controller to Week 11's navigation stack leans on it — while knowing not to trust it for long.

The Robot That Will Publish It



TurtleBot 3 Burger in Gazebo Harmonic. Two driven wheels, one caster, one axle — the mechanism from four slides ago, with a simulated floor underneath it that can slip. **From this week every lab runs here.**

Where the Error Comes From

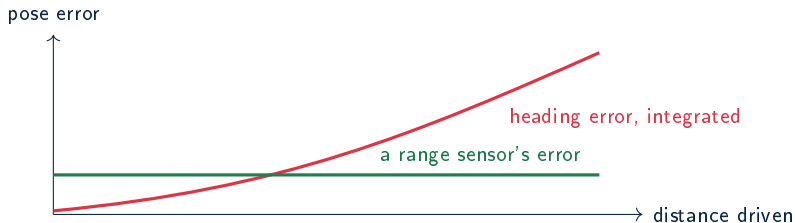
Source	Kind	Effect on the estimate
Unequal wheel radii	systematic	The robot curves while the model says straight. Constant bias, same direction every run.
Wheel separation L wrong	systematic	Every reported turn is scaled. 90° commanded, 93° driven, forever.
Average radius wrong	systematic	All distances scale. Tyre wear and payload both change it.
Wheel slip	non-systematic	Wheels turn, robot does not. Odometry gains distance that never happened.
Encoder resolution	non-systematic	Quantisation, bounded per step, but it never averages to zero once integrated.
Uneven floor, bumps	non-systematic	Momentary loss of contact. A carpet edge is worth several degrees.

Systematic errors are properties of the robot: measurable once, calibrated out, and they stay fixed.

Non-systematic errors are properties of the floor: not calibratable, and they are why odometry alone is never enough.

Borenstein & Feng, "Measurement and correction of systematic odometry errors in mobile robots," IEEE Trans. Robotics and Automation, 12(6), 1996 — the UMBmark procedure.

Why the Error Has No Ceiling



A small heading error does not stay small. It multiplies every metre driven after it: 1° of unnoticed heading error puts you 17 cm off after 10m, and it never recovers, because there is nothing in the chain that could notice.

Odometry error is **unbounded and monotone**. Every later week of this course is, in one way or another, about adding something that observes the world and pushes it back down.

Part 6

Open Loop, and What Comes Next

- 1 Pose in the Plane
- 2 Velocity
- 3 The Differential-Drive Model
- 4 From Velocity to Position
- 5 Wheel Odometry and Its Errors
- 6 Open Loop, and What Comes Next

Open Loop: The Whole Idea in One Line

To drive d metres: pick v , publish that `Twist` for

$$t = \frac{d}{v} \text{ seconds,} \quad \text{then publish zero.}$$

To turn ϕ radians: pick ω , publish for $t = \phi/\omega$, then publish zero.



no arrow comes back

Nothing measures the outcome. The robot is told what to do and never asked what happened. That missing arrow is the definition of open loop — and the entire subject of Week 5.

Open loop is not a mistake. It is the correct baseline, and you must see it fail before a controller means anything.

Where Open Loop Is Enough, and Where It Is Not

Works	Fails
Turtlesim — a perfect integrator with no floor, no slip and no mass.	Any real floor, over any distance worth driving.
Short moves: a nudge, a 10 cm reverse off a wall.	A square: four corners, four heading errors, and the path does not close.
Motions whose error the next step corrects anyway.	Anything repeated. The error compounds; it does not cancel.
Bringing a robot to a stop.	Reaching a specified pose.

In session B you will run the identical open-loop square in **both** places: it closes in turtlesim and it does not close in Gazebo. Nothing about the code changes. The floor changes.

Why turtlesim gets it right

Turtlesim *is* the integrator from three slides ago, run at 16 ms. There is no physics underneath it to disagree with. That is a feature for learning and a trap for trusting.

Session B Outline and Deliverables

- 1 Command motion from the terminal — straight, spin, arc — and watch the trail.
- 2 Write an open-loop `move_straight` on a timer, with a stop.
- 3 Write `rotate_in_place`, in radians, with the sign right.
- 4 Compose the two into a square, and measure the closing error from `/turtle1/pose`.
- 5 Run the same node against TurtleBot 3 in Gazebo Harmonic, reading `/odom`.
- 6 Report the two closing errors side by side.

Quiz 2 is this week — pose, Twist, the diff-drive equations, and the error table.

Next week the missing arrow is added: the robot is asked where it ended up, and told to fix it.